

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1023	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	Total Marks:	25
CO1 Statement:	Apply advanced methodologies and agile project management techniques to manage software projects effectively		
Student Name:	ARCHANA.T	Reg. No.:	192521295
Date:	07/10/2026	Duration:	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

- 1. Understand the Problem Statement**
 - Explain the problem in your own words.
 - Identify the objectives of the proposed system.
 - State the scope of the problem.
- 2. Identify Key Stakeholders and Challenges**
 - Identify all the stakeholders involved in the system.
 - Explain the roles and responsibilities of each stakeholder.
 - Discuss the major challenges and issues faced by the stakeholders.
- 3. Analyze the Current Approach**
 - Describe the existing system or current process.
 - Identify its strengths and limitations.
 - Analyze the problems or gaps that need to be addressed.
- 4. Propose a Suitable Solution Using Software Engineering Principles**
 - Design a software-based solution for the given problem.
 - Describe the major functional modules of the proposed system.
 - Identify the functional and non-functional requirements.
 - Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.
- 5. Justify the Chosen Methodology/Framework**
 - Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).

- Justify why the selected methodology/framework is suitable for the proposed system.
- Explain how it supports successful project development and maintenance.

6. Discuss Expected Outcomes and Limitations

- Describe the expected benefits of the proposed solution.
- Explain how the solution addresses the identified challenges.
- Discuss potential risks, implementation challenges, and limitations.
- Suggest possible future enhancements.

CASE STUDY ANALYSIS

AI-Assisted Software Development Governance and Reliability Platform

1. UNDERSTAND THE PROBLEM STATEMENT

1.1 Explanation of the Problem in My Own Words

Generative Artificial Intelligence and AI coding assistants are increasingly being used in modern software development. Developers can use these tools to generate source code, modify existing programs, create test cases, write documentation, suggest fixes and complete repetitive programming tasks. This can significantly improve productivity and reduce development time.

However, faster code generation does not automatically produce secure, correct or maintainable software. AI-generated code can contain programming defects, insecure coding practices, vulnerable dependencies, insufficient testing, duplicated logic, excessive complexity, outdated approaches or violations of organizational policies. A developer may also accept an AI-generated suggestion without completely understanding its implications.

The main problem is therefore how an organization can obtain the benefits of AI-assisted development without allowing unverified or unsafe changes to enter production. Traditional Git repositories, pull requests, code review, security tools and CI/CD pipelines provide useful controls, but the information can be distributed across different systems. This makes it difficult to maintain a complete view of AI involvement, risk, validation, approval and deployment.

The proposed AI-Assisted Software Development Governance and Reliability Platform (AISD-GRP) solves this problem by introducing a centralized governance layer across the software development life cycle. It captures AI-assisted code contributions, performs automated quality and security checks, identifies testing and dependency gaps, applies organizational policies, calculates risk, enforces human review for high-risk changes, maintains audit trails and integrates with CI/CD pipelines.

The platform follows a human-in-the-loop approach. AI is treated as an assistant rather than an authority. Automated tools perform repetitive checks, while qualified developers, security specialists and technical leads remain responsible for important engineering decisions.

1.2 Objectives of the Proposed System

No.	Objective	Purpose
1	Track AI-assisted changes	Record AI-assisted commits, pull requests and available provenance information.
2	Automate quality validation	Identify coding, complexity, duplication and maintainability problems.
3	Perform security validation	Detect insecure code, secrets and security vulnerabilities.
4	Analyze dependencies	Identify vulnerable, outdated or unauthorized third-party packages.
5	Validate testing	Execute automated tests and identify test coverage gaps.
6	Enforce policies	Apply configurable organizational quality, security and deployment

		rules.
7	Classify risk	Combine multiple indicators to identify Low, Medium and High risk changes.
8	Enforce human review	Require authorized approval for high-risk changes.
9	Maintain audit trails	Record code, test, review, deployment and rollback evidence.
10	Integrate CI/CD	Connect governance checks to build, test and deployment pipelines.
11	Monitor reliability	Track failures, deployment health, rollback events and delivery performance.
12	Support responsible AI	Promote transparency, accountability, privacy and ethical use of AI-generated code.

1.3 Scope of the Problem

The scope of the proposed system covers AI-assisted software changes from development through validation, review, deployment and post-deployment monitoring.

Area	Scope
Development	AI-assisted code contributions, commits and pull requests.
Version Control	Git/GitHub repository and pull-request integration.
Quality	Static analysis, complexity, duplication and maintainability checks.
Security	Security scanning, secret detection and vulnerability analysis.
Dependencies	Third-party package and dependency risk analysis.
Testing	Automated tests and test-coverage analysis.
Governance	Policy checking, risk scoring and approval workflow.
Delivery	CI/CD integration, deployment gates and deployment tracking.
Audit	Versioned records of changes, checks, approvals and deployments.
Reliability	Monitoring, failure detection and rollback support.
Reporting	Dashboards, governance reports and performance indicators.

The scope does not include developing a new foundation AI model, replacing software developers, replacing all existing development tools, or guaranteeing that every possible software defect will be detected.

1.4 Problem-to-Solution Mapping

Problem	Impact	Solution
Rapid AI code generation	Manual review may not be sufficient.	AI contribution tracking and automated validation.
Insecure generated code	Security weaknesses may reach production.	Security scanning and quality gates.
Vulnerable dependencies	Third-party risks can enter the application.	Dependency scanning.
Insufficient testing	New functionality may fail after deployment.	Automated tests and coverage analysis.
Policy violations	Code may not meet organizational standards.	Policy engine.
High-risk changes	Important changes may need expert attention.	Risk-based human review.
Fragmented evidence	Audits and investigations become difficult.	Centralized audit trail.
Production failures	Unreliable releases affect users.	Monitoring and rollback support.

2. IDENTIFY KEY STAKEHOLDERS AND CHALLENGES

2.1 Key Stakeholders, Roles and Responsibilities

Stakeholder	Roles and Responsibilities	Major Concern
Software Developers	Develop, modify, test and commit code; use AI responsibly; fix reported issues.	Fast feedback without excessive process overhead.
Tech Leads / Senior Developers	Review architecture, complex changes and technical debt; approve important changes.	Correctness, maintainability and architectural quality.
Security Team	Define security rules, investigate vulnerabilities and monitor security risks.	Secure code, secrets, dependencies and compliance.
QA / Test Engineers	Define testing requirements, execute tests and monitor coverage.	Regression prevention and adequate test coverage.
DevOps Engineers	Manage CI/CD, containers, deployment and rollback.	Stable and reliable software delivery.
Project Managers	Track scope, schedule, risk and delivery.	Predictable delivery and visibility.
Compliance / Audit Team	Review governance evidence and traceability.	Accountability and audit readiness.
System Administrators	Manage access, configuration and availability.	Platform security and uptime.
Management / Product Owners	Define priorities and risk tolerance.	Business value with acceptable risk.
AI Tool Providers	Provide AI coding capabilities and services.	Responsible use, privacy and service reliability.

2.2 Stakeholder Challenges

- AI-generated code can be produced faster than developers can manually inspect it.
- Security teams may face a larger number of findings due to rapid code generation.
- AI-generated functionality may not automatically have adequate tests.
- AI suggestions may introduce vulnerable or unnecessary third-party dependencies.
- Rapid development can increase technical debt and code complexity.
- High-risk changes require expert reviewers and can create review bottlenecks.
- Sensitive source code may require protection from unauthorized external AI services.
- Different repositories and CI/CD platforms may create integration difficulties.
- Audit teams require evidence of what changed, what checks were performed and who approved it.
- Production incidents can be difficult to investigate if AI contribution information is not recorded.

2.3 Stakeholder Interaction

Stakeholder	Input	Output / Benefit
Developer	Code, commit, pull request and AI-use information	Findings, risk score and remediation guidance
Security Team	Security policies and severity rules	Security findings and governance reports
QA Team	Testing standards and coverage requirements	Test and coverage results
Reviewer	Approval/rejection and comments	Recorded review decision
DevOps	Pipeline and deployment information	Release gate and deployment status
Management	Risk tolerance and KPIs	Governance dashboards
Auditor	Audit queries	Traceable evidence

3. ANALYZE THE CURRENT APPROACH

3.1 Existing System / Current Process

A typical current development process uses an AI coding assistant together with Git/GitHub, pull requests, code review, CI/CD and monitoring. The process is useful but governance information is often distributed across several tools.

1. The developer uses an AI coding assistant to generate or modify code.
2. The developer reviews the generated code and commits it to Git.
3. A pull request is created for collaboration and review.
4. CI/CD builds the application and may execute automated tests.
5. Security and quality checks may be performed using separate tools.
6. A reviewer approves or rejects the pull request.
7. The change is merged into the target branch.

8. The application is deployed to a test, staging or production environment.
9. Monitoring tools observe the deployed application.
10. If a serious failure occurs, the team initiates a rollback.

The major weakness is that there may be no single governance layer connecting AI provenance, quality, security, testing, risk, approval, deployment and audit evidence.

3.2 Strengths of the Existing Approach

Strength	Explanation
Version Control	Git maintains source-code history and supports collaboration.
Pull Requests	Developers can review changes before merging.
CI/CD	Build, test and deployment activities can be automated.
AI Productivity	Coding assistants can reduce development effort.
Automated Testing	Testing frameworks can detect many functional defects.
Security Tools	Specialized tools can scan source code and dependencies.
Monitoring	Operational tools can identify production failures.

3.3 Limitations and Gaps

Limitation	Effect	Priority
Incomplete AI provenance	Difficult to determine which changes were AI-assisted.	High
Fragmented validation	No single risk-based view of all checks.	High
Non-risk-based review	High-risk changes may not receive additional attention.	High
Dependency risk	Vulnerable packages may enter the system.	High
Testing gaps	Generated code may lack adequate tests.	High
Distributed audit evidence	Investigations take more time.	Medium
Technical debt	Rapid generation can increase complexity and duplication.	High
Deployment instability	Poorly validated changes may fail in production.	High
Privacy risk	Sensitive source code may be exposed to unauthorized services.	High

4. PROPOSE A SUITABLE SOLUTION USING SOFTWARE ENGINEERING PRINCIPLES

4.1 Proposed Solution

The proposed AISD-GRP acts as a governance and reliability layer between the source repository and the delivery pipeline. When a developer creates or updates a pull request, the platform captures the change and performs automated validation.

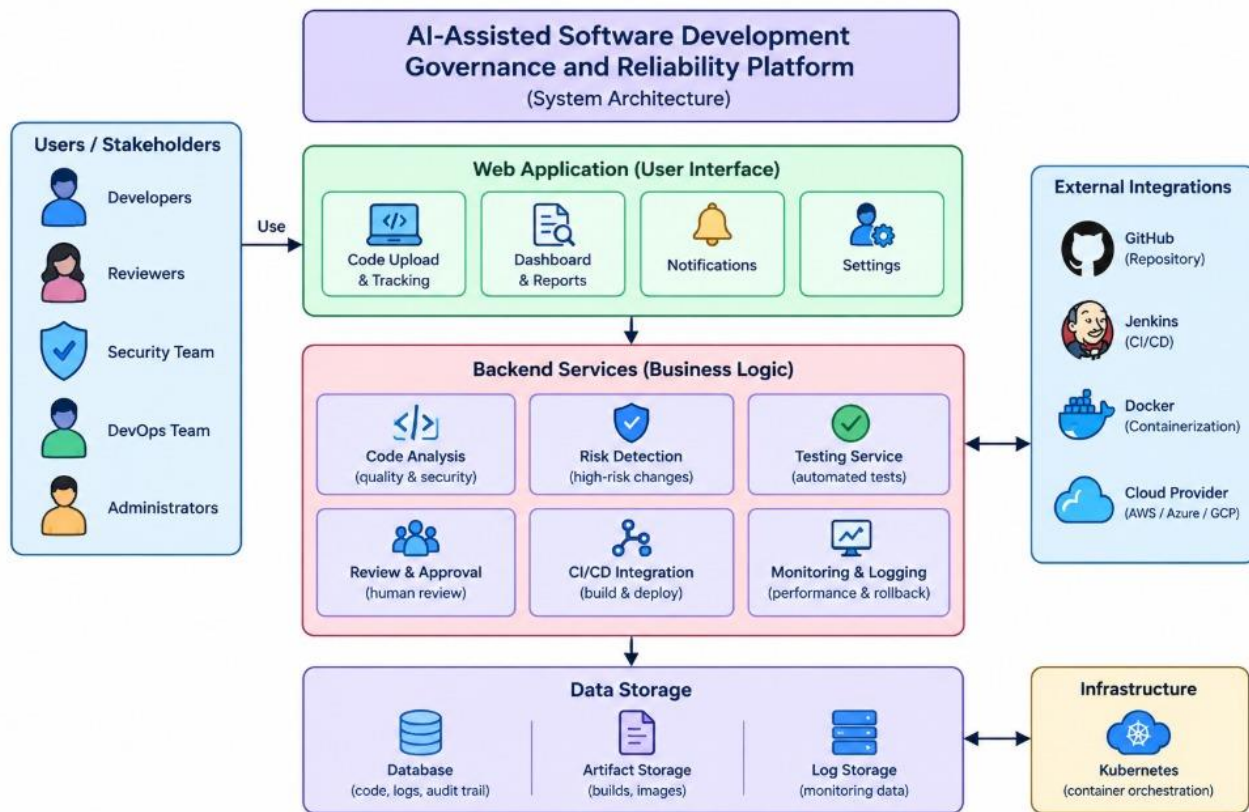
The platform checks code quality, security, dependencies, testing, coverage and organizational policies. It then combines the evidence into a risk classification. Low-risk changes can continue when all mandatory gates pass. High-risk changes are blocked and routed to an authorized human reviewer. After approval, the change continues through CI/CD. Deployment status is monitored and rollback evidence is recorded when required.

The solution therefore provides a complete traceability chain: Code Change → Automated Validation → Risk Assessment → Human Review → CI/CD → Deployment → Monitoring → Audit.

4.2 Major Functional Modules

Module	Detailed Functions
1. User & Access Management	Authentication, role-based access control, reviewer authorization and secure session management.
2. AI Contribution Tracker	Capture AI-assisted commit and pull-request metadata where available.
3. Code Quality Analyzer	Check coding standards, complexity, duplication and maintainability.
4. Security & Dependency Scanner	Detect vulnerabilities, secrets, unsafe patterns and dependency risks.
5. Test & Coverage Manager	Run automated tests, store results and identify coverage gaps.
6. Policy Engine	Apply configurable quality, security, testing and deployment rules.
7. Risk Engine	Calculate risk from findings, change context, coverage and sensitivity.
8. Human Review Workflow	Assign high-risk changes to authorized reviewers and record decisions.
9. CI/CD Integration	Connect governance gates with build, test and deployment pipelines.
10. Audit & Compliance	Maintain versioned records of changes, checks, approvals and deployments.
11. Monitoring Dashboard	Display pipeline health, deployment status, failures and rollback events.
12. Reporting	Generate governance, security, quality and reliability reports.

4.3 System Architecture Diagram



4.4 Detailed System Workflow

1. Developer creates or modifies code using an AI coding assistant where permitted.
2. Developer pushes the change to Git/GitHub and creates or updates a pull request.
3. A webhook informs the governance platform about the repository event.
4. The AI Contribution Tracker records available AI-assistance and change metadata.
5. The Code Quality Analyzer checks complexity, duplication, maintainability and coding standards.
6. The Security Scanner checks source code for security problems and exposed secrets.
7. The Dependency Scanner checks third-party libraries for known vulnerabilities and policy restrictions.
8. The Test Manager executes automated tests.
9. The Coverage Manager calculates test coverage and identifies important untested areas.
10. The Policy Engine evaluates organizational governance rules.
11. The Risk Engine combines findings and assigns Low, Medium or High risk.
12. Low-risk changes can continue when mandatory gates pass.
13. High-risk changes are blocked and assigned to an authorized human reviewer.
14. The reviewer approves, rejects or requests remediation.
15. Approved changes continue through the CI/CD pipeline.
16. The application is deployed to the selected environment.
17. Monitoring records deployment health and operational behavior.
18. If a serious failure occurs, rollback can be initiated according to release policy.

19. All major events are stored in the audit trail.

4.5 Risk Classification

Risk	Typical Conditions	Action
Low	Small change, tests pass, acceptable coverage and no critical findings.	Automated progression if all required gates pass.
Medium	Moderate complexity, coverage gap, minor vulnerability or dependency concern.	Additional validation and/or reviewer confirmation.
High	Critical security finding, sensitive component, major architecture change, policy violation or critical test failure.	Block deployment and require remediation plus authorized human approval.

4.6 Functional Requirements

ID	Functional Requirement
FR-01	The system shall authenticate users securely.
FR-02	The system shall enforce role-based access control.
FR-03	The system shall record available AI-assisted contribution information.
FR-04	The system shall integrate with Git/GitHub repositories.
FR-05	The system shall monitor pull-request events.
FR-06	The system shall perform automated code-quality analysis.
FR-07	The system shall perform security scanning.
FR-08	The system shall scan software dependencies.
FR-09	The system shall execute automated tests.
FR-10	The system shall calculate test coverage.
FR-11	The system shall evaluate configurable organizational policies.
FR-12	The system shall calculate a risk classification.
FR-13	The system shall block high-risk changes without required approval.
FR-14	The system shall provide a human review workflow.
FR-15	The system shall integrate with CI/CD pipelines.
FR-16	The system shall maintain versioned audit records.
FR-17	The system shall record deployment and rollback events.
FR-18	The system shall provide dashboards and reports.

4.7 Non-Functional Requirements

Attribute	Requirement
-----------	-------------

Security	Protect source code, credentials, audit data and user information using appropriate security controls.
Reliability	Provide consistent validation and graceful failure handling.
Scalability	Support increasing repositories, users and analysis workloads.
Performance	Provide validation feedback within an acceptable development workflow time.
Availability	Keep governance services available for CI/CD operations.
Maintainability	Use modular services, clean APIs, automated tests and documentation.
Usability	Present findings and recommended actions clearly.
Auditability	Record timestamps, users, versions, decisions and validation evidence.
Privacy	Minimize exposure of confidential source code and sensitive information.
Portability	Support containerized deployment in cloud or on-premises environments.
Interoperability	Integrate with repository, CI/CD, security and testing tools.

4.8 Software Engineering Principles Applied

Principle	Implementation	Benefit
Modularity	Separate provenance, quality, security, testing, risk, review, audit and monitoring modules.	Easy maintenance and independent development.
Scalability	Use containers and independently scalable analysis workers.	Supports increased workload.
Maintainability	Use documented APIs, configuration-driven policies and automated tests.	Simplifies modifications.
Reliability	Use quality gates, monitoring, health checks and rollback support.	Improves release stability.
Usability	Provide dashboards with risk, findings and recommended actions.	Makes the system easy to understand.
Security	Use RBAC, encryption, least privilege, secret management and security scanning.	Protects the platform and software.
Traceability	Link change, checks, risk, review, deployment and outcome.	Provides accountability.
Separation of Concerns	Keep analysis, policy, audit and deployment responsibilities separate.	Reduces coupling.
Testability	Use unit, integration, security and	Improves confidence.

	pipeline tests.	
Reusability	Use reusable policy and validation services across repositories.	Reduces duplicated development effort.

4.9 Suggested Technology Stack

Layer	Suggested Technology	Purpose
Frontend	React / HTML / CSS / JavaScript	Dashboard and reviewer interface.
Backend	Python FastAPI/Flask	REST APIs and business logic.
Database	PostgreSQL	Production data and audit storage.
Prototype Database	SQLite	Simple academic prototype.
Version Control	Git / GitHub	Source management and collaboration.
CI/CD	GitHub Actions or equivalent	Automated build, test and deployment.
Containerization	Docker	Portable deployment.
Testing	Pytest / JUnit	Automated testing.
Monitoring	Prometheus / Grafana or equivalent	Metrics and monitoring.
Authentication	OAuth / OIDC / SSO	Secure identity management.
Cloud	AWS / Azure / Google Cloud	Scalable infrastructure.

4.10 Security and Ethical AI Considerations

- AI assistance should be transparent when organizational policy requires disclosure.
- Sensitive source code should not be sent to external AI services without authorization.
- Credentials, passwords, API keys and secrets must never be exposed through prompts or logs.
- High-risk engineering decisions should remain subject to qualified human oversight.
- Audit records should explain what checks were performed and who approved the change.
- The platform should never assume that AI-generated code is automatically correct.
- Access to source code and audit information should follow least-privilege principles.
- Governance rules should be clearly documented so developers understand why a change is blocked.
- AI-generated code should undergo the same engineering validation expected of other software changes.

5. JUSTIFY THE CHOSEN METHODOLOGY / FRAMEWORK

5.1 Selected Methodology: Agile Scrum with DevSecOps

The recommended approach is Agile Scrum combined with DevSecOps practices. Scrum is suitable because AI governance requirements can evolve rapidly as organizations adopt new AI coding assistants, discover new security risks and refine their development policies. The platform should therefore be developed incrementally with frequent stakeholder feedback.

DevSecOps is combined with Scrum because security, testing, governance and reliability should be integrated continuously rather than performed only at the end of development. This makes the methodology especially appropriate for a platform whose main purpose is secure and reliable software delivery.

5.2 Justification for Agile Scrum

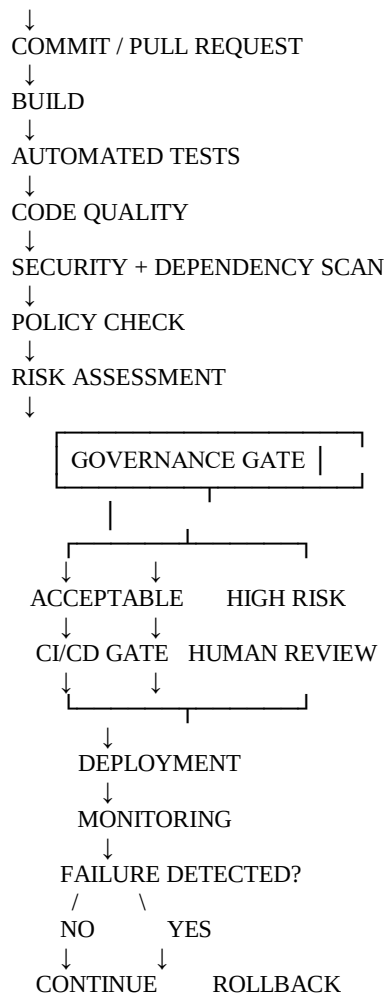
Reason	Justification
Changing requirements	AI tools and governance policies can change frequently.
Incremental development	Authentication, tracking, scanning, risk, review and monitoring can be delivered separately.
Continuous feedback	Developers, security, QA and DevOps teams can review working features each sprint.
Early risk detection	Security and architecture issues can be identified before final delivery.
Continuous improvement	Retrospectives can improve governance rules and user experience.
Stakeholder involvement	Product owners and technical stakeholders can prioritize useful features.
CI/CD compatibility	Automated testing, scanning and deployment fit naturally into iterative development.

5.3 Proposed Sprint Plan

Sprint	Activities	Expected Deliverable
1	Requirements, stakeholders, architecture and backlog.	Approved requirements and architecture.
2	Authentication, RBAC and basic dashboard.	Working access-management module.
3	Git/GitHub integration and AI contribution tracking.	Tracked repository changes.
4	Code-quality, security and dependency scanning.	Automated validation.
5	Testing, coverage and policy engine.	Testing and policy gates.
6	Risk scoring and human-review workflow.	Risk-based approval.
7	CI/CD, deployment and rollback support.	Controlled delivery.
8	Monitoring, hardening, performance and UAT.	Integrated final system.

5.4 DevSecOps Pipeline

PLAN
↓
CODE + AI ASSISTANCE



5.5 How the Methodology Supports Development and Maintenance

- New requirements can be placed in the product backlog and prioritized.
- Modules can be implemented incrementally instead of waiting for the entire system.
- Automated tests can run during every sprint.
- Security checks can be integrated continuously through DevSecOps.
- Stakeholders can provide feedback after each sprint review.
- Technical debt can be tracked and planned as future backlog work.
- Production issues can be converted into new improvement tasks.
- Retrospectives help the team improve its development process.
- Continuous integration reduces integration and regression problems.
- Continuous monitoring provides feedback after deployment.

6. DISCUSS EXPECTED OUTCOMES AND LIMITATIONS

6.1 Expected Benefits

Expected Outcome	How the Platform Achieves It
Reduced insecure AI-generated code	Security and dependency scanning plus risk-based gates.

Improved software quality	Static analysis, testing and quality thresholds.
Better testing	Automated tests and coverage measurement.
Improved accountability	AI contribution, review and deployment records.
Better traceability	Complete link between changes, checks, reviews and releases.
Reliable delivery	CI/CD gates, monitoring and rollback support.
Reduced technical debt	Complexity, duplication and maintainability checks.
Efficient human review	Review effort is focused on higher-risk changes.
Better governance	Centralized and configurable organizational policies.
Responsible AI adoption	Transparency, privacy and human oversight.

6.2 Key Performance Indicators

KPI	Purpose
AI-assisted changes with recorded provenance (%)	Measure traceability.
Changes passing automated quality gates (%)	Measure quality effectiveness.
High-risk changes reviewed before deployment (%)	Measure governance compliance.
Critical security findings blocked	Measure security prevention.
Average test coverage (%)	Measure testing maturity.
Change failure rate	Measure delivery reliability.
Mean Time to Recovery (MTTR)	Measure recovery performance.
Rollback frequency	Identify release instability.
Technical-debt trend	Measure maintainability.
Average validation time	Measure platform performance.

6.3 Risks and Implementation Challenges

Risk / Challenge	Impact	Mitigation
False positives	Developers may lose trust in the platform.	Tune rules and allow justified suppression.
Slow validation	Developer productivity may decrease.	Parallelize checks, cache results and scale workers.
Incomplete AI provenance	Some AI-assisted changes may not be identified.	Use available metadata and document limitations.
Source-code privacy	Confidential information could be exposed.	Use encryption, access control and secure integrations.
Reviewer overload	High-risk queue can become a bottleneck.	Prioritize by severity and distribute review work.
Integration failure	CI/CD may be interrupted.	Use retries, health checks and clear failure policies.
Complex policies	Rules may become difficult to manage.	Use versioned configuration-driven policies.
False sense of security	Users may assume automation detects every defect.	Communicate limitations and retain human judgment.
Infrastructure cost	Large repositories may require high compute capacity.	Use scalable workers and caching.

6.4 Limitations

- Automated analysis cannot guarantee that software is completely secure or defect-free.
- Risk scoring supports decision-making but cannot replace expert engineering judgment.
- AI contribution tracking depends on the metadata available from the AI tool and development environment.
- Organizations may require different risk thresholds and governance policies.
- Large repositories may require substantial processing resources.
- Different Git providers and CI/CD systems may require custom integrations.
- Unknown vulnerabilities may not be detected by existing tools.
- The platform cannot completely eliminate human errors or malicious activity.
- Some compliance and ethical decisions require organizational or legal expertise.
- Governance rules require continuous tuning to balance security and developer productivity.

6.5 Future Enhancements

1. Explainable AI risk analysis that provides clear reasons for risk classifications.
2. IDE integration to provide governance feedback before pull-request submission.
3. Software Bill of Materials (SBOM) generation and continuous supply-chain monitoring.
4. Predictive reliability analytics based on historical deployments and incidents.
5. Automated technical-debt prioritization based on engineering impact.
6. Privacy-preserving AI analysis for sensitive repositories.
7. Anomaly detection for unusual repository and deployment behavior.
8. Support for additional Git providers and enterprise CI/CD platforms.
9. Automated compliance reports for internal audits and regulated environments.
10. Advanced AI-use policies that detect prohibited or inappropriate AI-assisted practices.

6.6 How the Proposed Solution Addresses the Overall Problem

The proposed platform directly addresses the central challenge of AI-assisted software development by placing verification between rapid code generation and production deployment. Instead of trusting or rejecting AI completely, the organization can use a risk-based approach. Low-risk changes can move quickly through automated checks, while high-risk changes receive additional human attention.

This approach preserves developer productivity while improving quality, security and accountability. The centralized audit trail also means that an organization can investigate a change after deployment and determine the repository version, validation results, review decision and deployment outcome associated with it.

CONCLUSION

The AI-Assisted Software Development Governance and Reliability Platform is a suitable software engineering solution for organizations that increasingly depend on generative AI and AI coding assistants. AI can improve development productivity, but uncontrolled AI-generated changes may

introduce security vulnerabilities, poor testing, policy violations, technical debt and operational instability.

The proposed platform addresses these issues through AI contribution tracking, automated code-quality analysis, security and dependency scanning, testing and coverage analysis, policy enforcement, risk classification, human approval, CI/CD integration, audit trails and deployment monitoring.

The design applies important software engineering principles including modularity, scalability, maintainability, reliability, usability, security, separation of concerns, testability and traceability. The human-in-the-loop approach ensures that automation supports developers and reviewers rather than removing professional responsibility.

Agile Scrum combined with DevSecOps is appropriate because the platform must evolve continuously with AI technologies, security threats, organizational policies and development practices. Incremental sprints, continuous testing, stakeholder feedback and integrated security provide a practical approach for successful development and long-term maintenance.

Overall, the proposed platform enables organizations to adopt AI-assisted software development in a controlled and responsible way. Its main goal is not to stop developers from using AI, but to ensure that AI-assisted software is validated, reviewed, traceable, secure and reliable before it reaches production.

FINAL ASSESSMENT SUMMARY

Rubric Area	How the Report Meets the Excellent Level
Problem Understanding – 30%	Detailed explanation of the problem, objectives, scope, boundaries and problem-to-solution mapping.
Stakeholder & Challenge Analysis – 25%	Identifies major stakeholders, responsibilities, interactions and challenges.
Current System Analysis – 20%	Explains the existing process, strengths, limitations and detailed gaps.
Proposed Solution & Engineering Principles – 15%	Provides architecture, modules, requirements, technology stack and application of software engineering principles.
Methodology / Framework – 10%	Selects Agile Scrum with DevSecOps and provides technical justification, sprint plan and maintenance approach.

REFERENCES

1. I. Sommerville, Software Engineering, 10th Edition, Pearson.
2. R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, McGraw-Hill.
3. National Institute of Standards and Technology (NIST), Secure Software Development Framework (SSDF), NIST Special Publication 800-218.
4. OWASP Foundation, Software Security and Secure Development Resources.

5. GitHub Documentation, GitHub Actions, Repository and Pull Request Documentation.
6. Docker Documentation, Containerization and Deployment Documentation.
7. DORA / Google Cloud, Software Delivery Performance and Reliability Research.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are

				largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/framework and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria	Mark
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25